



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатики и системы управления (ИУ)»

КАФЕДРА «Информационные системы и телекоммуникации (ИУЗ)»

ОТЧЕТ
по лабораторным работам 2–4
по курсу «РПО»

Студент ИУЗ-63Б
(Группа)

(Подпись, дата)

Абаничев А. И.
(И. О. Фамилия)

Преподаватель

(Подпись, дата)

Сидякин И. М.
(И. О. Фамилия)

2026 г.

СОДЕРЖАНИЕ

1	Введение	3
1.1	Архитектура	3
1.2	Стандарт MIFARE Classic и ридер PN532	4
1.3	Схема данных на карте	4
2	Проектирование	5
2.1	База данных	5
2.2	API бэкенда	5
2.3	Фронтенд	6
2.4	Клиент. Взаимодействие с аппаратурой	7
2.5	Фрагменты кода	8
3	Тестирование	11
4	Заключение	12
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	13

1 Введение

В лабораторных работах 2–4 я собрал систему для авторизации платежей транспортными картами. Серверная часть (ЛР2) хранит данные и принимает решение по операциям; веб-интерфейс (ЛР3) нужен для управления и просмотра логов; NFC-терминал (ЛР4) работает с физической картой MIFARE Classic и обменивается с сервером по HTTPS.

Цель работы — связать сервер, админку и NFC-ридер в один сценарий: от регистрации карты до отображения операций в панели администратора.

1.1 Архитектура

Проект разделён на три части. Центральный узел — REST API на Go [1]; перед ним стоит nginx с TLS, всё запускается через Docker [2]. Данные хранятся в SQLite [3], миграции применяются при старте сервера. Для пользователей админки и терминалов используются разные JWT [4].

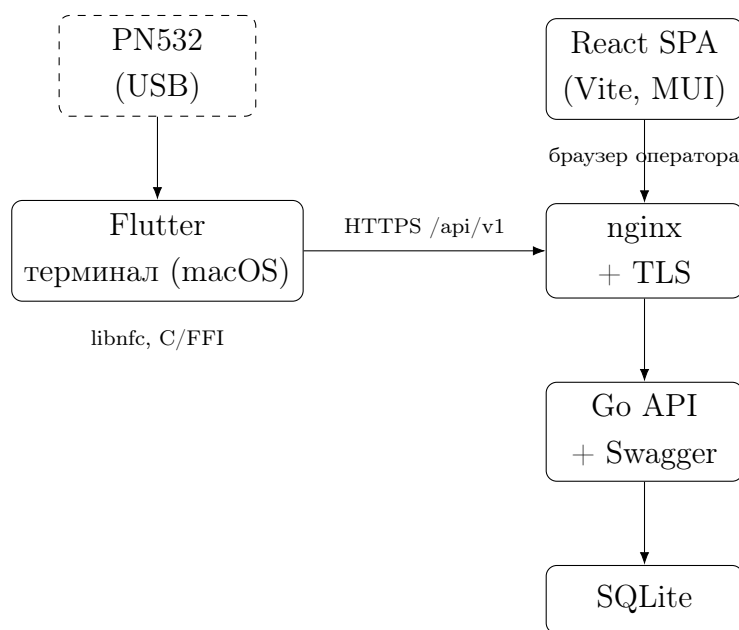


Рисунок 1.1 – Архитектура проекта РПО (ЛР2–4)

Веб-клиент — SPA на React и Material UI: в нём доступны карты, терминалы, ключи, пользователи, транзакции и журнал событий. Терминал — приложение Flutter для macOS; через FFI оно вызывает C-код, который работает с PN532 через libnfc [5]. Сервер публикует Swagger UI на `/api/v1/swagger/`.

При списании терминал читает JSON с карты и отправляет `POST /terminal/authorize`. Если сервер разрешает операцию, терминал умень-

шает баланс на карте и отправляет событие `debit_card`; сервер в той же логике обновляет баланс в БД и создаёт транзакцию. Пополнение проходит аналогично через событие `credit_balance`.

1.2 Стандарт MIFARE Classic и ридер PN532

MIFARE Classic — бесконтактные карты стандарта ISO/IEC 14443 Type A [6]. Память разбита на сектора по 4 блока по 16 байт, доступ защищён ключами A/B. В работе используется карта на 1 Кбайт.

PN532 — NFC-контроллер NXP для работы с картами [7]. В моей установке ридер подключён по USB, а обмен командами MIFARE выполняет `libnfc`.

1.3 Схема данных на карте

Данные карты занимают шесть блоков секторов 1 и 2 (блоки 4–6 и 8–10). Каждый 16-байтный блок перед записью шифруется AES-128-ECB с помощью OpenSSL [8]; AES-ключ получается из MIFARE Key A, который хранится на сервере в таблице `keys`. После расшифровки данные имеют вид:

```
{"v":1,"card_number":"A1B2C3D4","balance":1000,"key_id":1}
```

Поле `card_number` при регистрации совпадает с UID карты; `key_id` связывает запись с ключом шифрования в БД.

до 96 байт JSON → AES-128-ECB по блокам



Рисунок 1.2 – Расположение данных на MIFARE Classic

2 Проектирование

2.1 База данных

Схема БД описана SQL-миграциями `goose`. Основные таблицы: `users`, `keys`, `terminals`, `cards`, `transactions`, `terminal_events`.

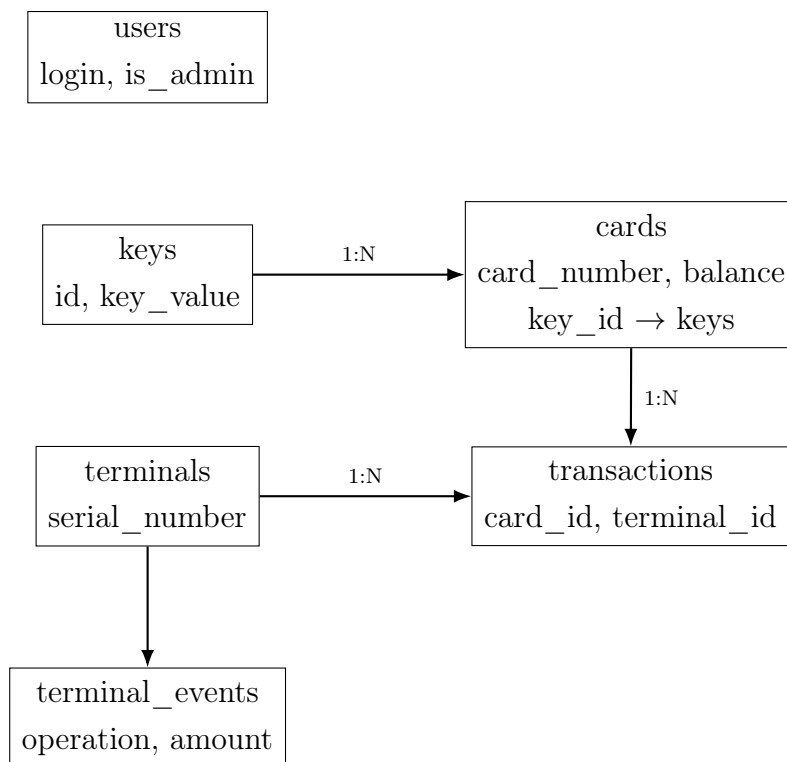


Рисунок 2.1 – Логическая схема связей таблиц SQLite

`cards` хранит номер карты, баланс, флаг блокировки и ссылку на ключ. `transactions` фиксирует сумму, карту и терминал. При событиях `debit_card` и `credit_balance` баланс обновляется внутри одной транзакции БД. `terminal_events` используется для журнала терминала; SPA получает новые записи через `GET /terminal/events?since=` каждые 2 с.

2.2 API бэкенда

REST API версии `v1` документирован в Swagger [9]. Базовый префикс: `https://localhost:8888/api/v1`. Основные группы эндпоинтов приведены в таблице 2.1.

Таблица 2.1 – Основные группы API

Группа	Назначение
auth	POST /auth/login — JWT пользователя; GET /me — профиль
terminal	POST /terminal/auth/login; POST /terminal/authorize (решение без списания); GET /terminal/keys; POST /terminal/register-card; POST /terminal/event; GET /terminal/events
cards, terminals, transactions	CRUD для авторизованных пользователей
keys, users	CRUD только для администратора

/terminal/authorize проверяет терминал, карту, блокировку и достаточность баланса, возвращая {approved, reason}. Фактическое изменение баланса выполняется при событии debit_card; так сервер остаётся арбитром, а карта хранит локальную копию баланса.

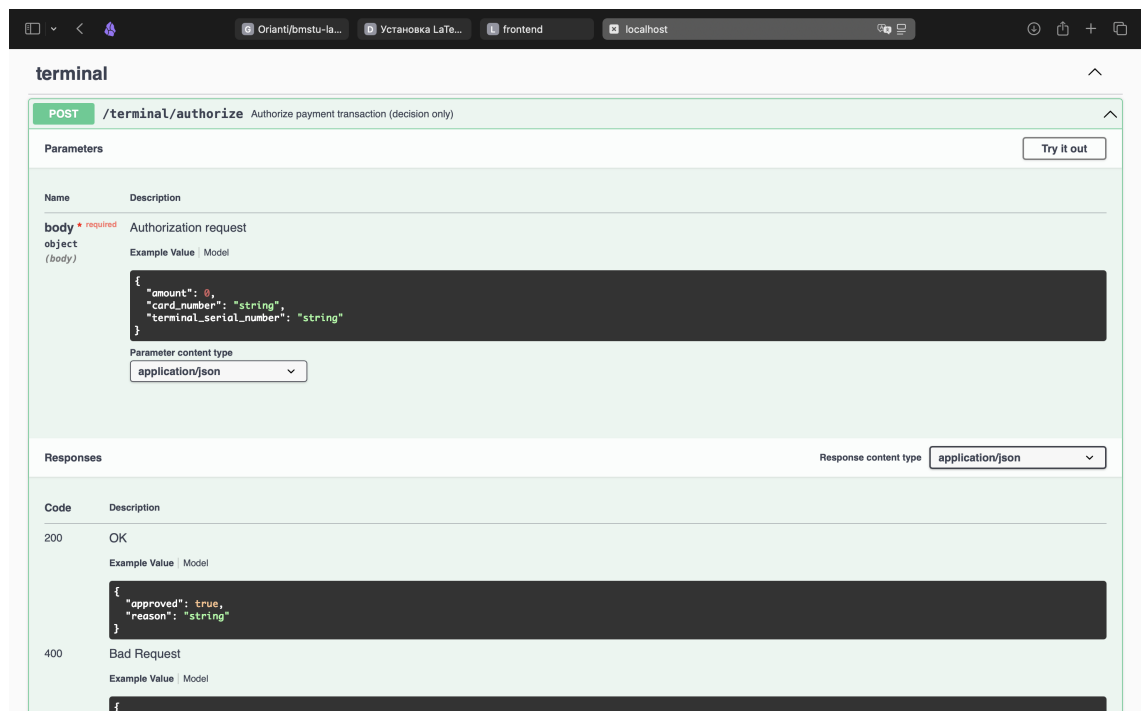


Рисунок 2.2 – Описание эндпоинта авторизации в Swagger UI

2.3 Фронтенд

Клиентская часть написана на React и TypeScript, сборка выполняется Vite, интерфейс собран на Material UI [10; 11]. Есть страница входа /login и защищённая зона с боковым меню: карты, терминалы, транзакции, ключи и пользователи. CrudTable и EntityDialog используются на большинстве стра-

ниц, поэтому таблицы и формы не дублируются вручную. Структура экранов показана на рис. 2.3; на странице «Терминалы» отдельно показан live-журнал NFC-операций (рис. 2.4). Для администратора владелец карты выбирается из списка пользователей, обычный пользователь вводит имя вручную.

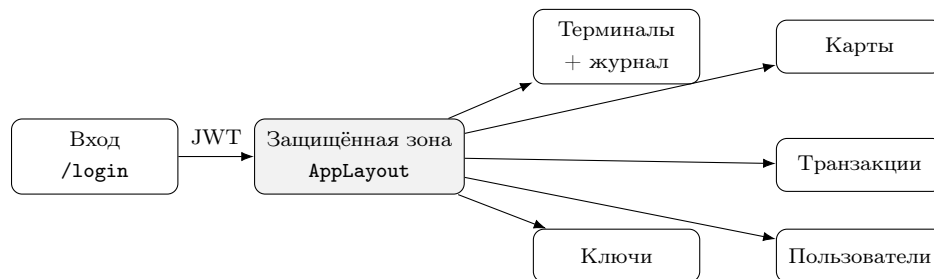


Рисунок 2.3 – Структура экранов веб-интерфейса

Транспортные карты						admin (admin) SWAGGER Выход
Терминалы	Терминалы					+ ДОБАВИТЬ
Карты	ID	SN	Адрес	Название	Создан	Действия
Транзакции	2	TERM-MAC-001	bmstu		2026-05-25T11:10:45Z	
Ключи	1	123	af	asdf	2026-05-25T09:55:44Z	
Пользователи	Журнал терминала					
	Причина / время	Операция	SN	Карта	Сумма	ОК
	2026-05-29T12:42:56Z	credit_balance	TERM-MAC-001	89FC2006	1000	—
	2026-05-29T12:42:46Z	debit_card	TERM-MAC-001	89FC2006	100	—
	2026-05-29T12:42:45Z — ok	authorize	TERM-MAC-001	89FC2006	100	Да
	2026-05-29T12:42:31Z	register_card	TERM-MAC-001	89FC2006	1000	—
	2026-05-29T12:28:28Z	credit_balance	TERM-MAC-001	89FC2006	1000	—
	2026-05-29T12:28:18Z	credit_balance	TERM-MAC-001	89FC2006	100	—
	2026-05-29T12:28:12Z	debit_card	TERM-MAC-001	89FC2006	100	—
	2026-05-29T12:28:11Z — ok	authorize	TERM-MAC-001	89FC2006	100	Да
	2026-05-29T12:28:00Z	register_card	TERM-MAC-001	89FC2006	1000	—
	2026-05-29T12:12:10Z	credit_balance	TERM-MAC-001	89FC2006	1000	—
	2026-05-29T12:12:01Z	credit_balance	TERM-MAC-001	89FC2006	100	—
	2026-05-29T12:11:53Z	debit_card	TERM-MAC-001	89FC2006	100	—

Рисунок 2.4 – Фрагмент веб-интерфейса

2.4 Клиент. Взаимодействие с аппаратурой

Терминальное приложение написано на Flutter и запускается на macOS. Работа с ридером вынесена в C-модуль `nfc_card.c`; он линкуется в Runner и вызывается из Dart через FFI [12].

Подключение библиотек:

1. установлены зависимости Homebrew: `libnfc`, `openssl@3`;

2. в `NfcNative.xcconfig` указаны `include/lib` пути и флаги `-lnfc -lssl -lcrypto`;
3. `nfc_card.c` и `mifare_compat.c` добавлены в macOS Runner;
4. `NfcBridge` загружает функции `nfc_read_card`, `nfc_write_card`, `nfc_reader_present`;
5. ожидание карты выполняется в `Isolate.run`, чтобы интерфейс не зависал.

В терминале есть вкладки «Регистрация», «Карта», «Списание», «Пополнение». При старте приложение логинится через `POST /terminal/auth/login`, загружает ключи с `GET /terminal/keys` и проверяет ридер функцией `nfc_reader_present()`.

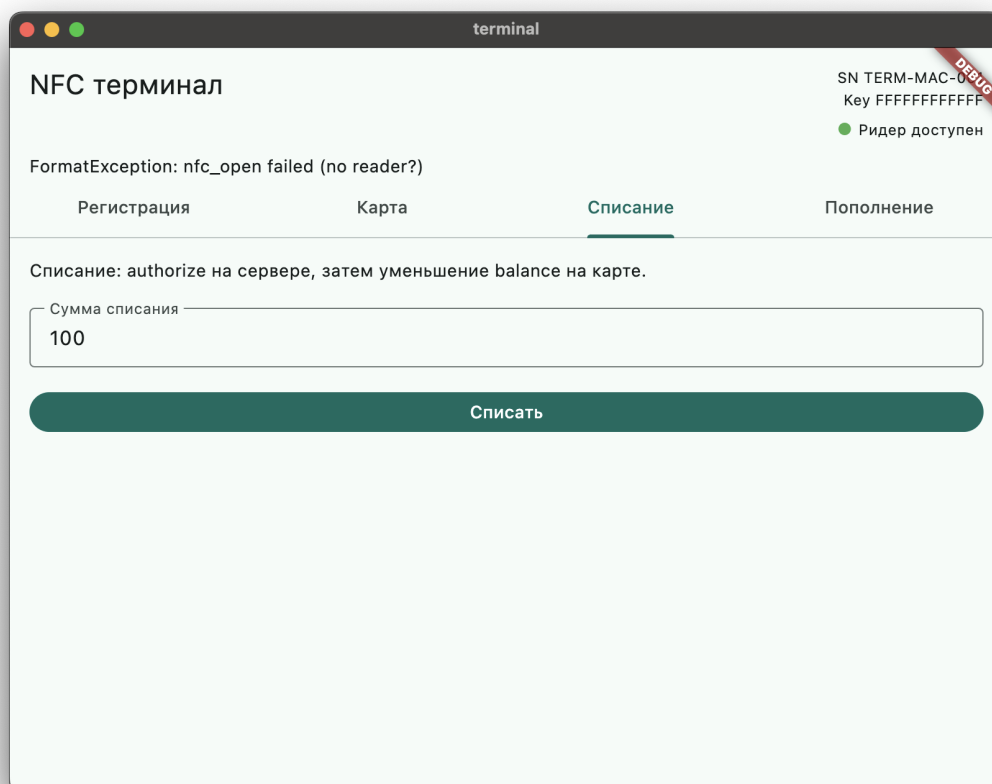


Рисунок 2.5 – Интерфейс NFC-терминала

2.5 Фрагменты кода

Проверка платежа на сервере:

```

if _, err := (store.Terminals{DB: s.DB}).GetBySerialNumber(
    r.Context(), req.TerminalSerialNumber); err != nil {
    writeError(w, http.StatusInternalServerError, "internal",
        "database error")
    return
}

card, err := (store.Cards{DB:
    s.DB}).GetByCardNumber(r.Context(), req.CardNumber)
if err != nil {
    writeError(w, http.StatusInternalServerError, "internal",
        "database error")
    return
}

if card.IsBlocked {
    resp = terminalAuthorizeResponse{Approved: false, Reason:
        "card_blocked"}
} else if card.Balance < req.Amount {
    resp = terminalAuthorizeResponse{Approved: false, Reason:
        "insufficient_funds"}
} else {
    resp = terminalAuthorizeResponse{Approved: true, Reason:
        "ok"}
}

```

Запись блоков MIFARE с аутентификацией и шифрованием:

```

uint8_t aes_key[16];
derive_aes_key(mifare_key, aes_key);

for (int i = 0; i < DATA_BLOCK_COUNT; i++) {
    uint8_t block = k_data_blocks[i];
    if (mifare_auth(session->dev, nt, block, mifare_key) !=
        NFC_CARD_OK) {
        return NFC_CARD_ERR;
    }
    mifare_param mp;
    uint8_t wire[BLOCK_SIZE];
    const uint8_t *src = plain + i * BLOCK_SIZE;
    if (encrypt_block_aes(aes_key, src, wire) != NFC_CARD_OK) {
        return NFC_CARD_ERR;
    }
}

```

```

    }
    memcpy(mp.mpd.abtData, wire, BLOCK_SIZE);
    if (!nfc_initiator_mifare_cmd(session->dev, MC_WRITE, block,
        &mp)) {
        set_error("mifare write failed");
        return NFC_CARD_ERR;
    }
}

```

Вызов C-функции из Dart через FFI:

```

final keyPtr = _allocUtf8(mifareKeyHex);
final uidBuf = calloc<ffi.Uint8>(64);
final jsonBuf = calloc<ffi.Uint8>(1024);
try {
    final rc = _nfcReadCard(keyPtr, uidBuf, 64, jsonBuf, 1024);
    if (rc != 0) {
        throw FormatException(lastLibError());
    }
    return (
        uid: _peekUtf8Zeros(uidBuf, 64),
        json: _peekUtf8Zeros(jsonBuf, 1024),
    );
} finally {
    calloc.free(keyPtr);
    calloc.free(uidBuf);
    calloc.free(jsonBuf);
}

```

Регистрация терминальных маршрутов API:

```

mux.Handle("POST /api/v1/terminal/authorize",
    terminalAuthed(s.handleTerminalAuthorize))
mux.Handle("GET /api/v1/terminal/keys",
    terminalAuthed(s.handleTerminalKeys))
mux.Handle("POST /api/v1/terminal/event",
    terminalAuthed(s.handleTerminalEventPost))
mux.Handle("POST /api/v1/terminal/register-card",
    terminalAuthed(s.handleTerminalRegisterCard))

```

3 Тестирование

Функциональное тестирование я выполнял вручную: сервер запускался через Docker Compose, терминал — через Flutter на macOS.

1. **Развёртывание:** `docker compose up -build`, затем проверка `/health` и Swagger.
2. **Админка:** вход `admin/admin`, создание ключа MIFARE и терминала TERM-MAC-001.
3. **Терминал:** `flutter run -d macos`, регистрация карты и чтение баланса.
4. **Списание:** успешный `debit` при согласованном балансе карты и БД.
5. **Отказ:** сумма выше баланса возвращает `approved=false` и `insufficient_funds`.
6. **Пополнение:** `credit_balance` увеличивает баланс на карте и в БД.

Журнал терминала

Причина / время	Операция	SN	Карта	Сумма	ОК
2026-05-29T12:42:56Z	credit_balance	TERM-MAC-001	89FC2006	1000	—
2026-05-29T12:42:46Z	debit_card	TERM-MAC-001	89FC2006	100	—
2026-05-29T12:42:45Z — ok	authorize	TERM-MAC-001	89FC2006	100	Да
2026-05-29T12:42:31Z	register_card	TERM-MAC-001	89FC2006	1000	—
2026-05-29T12:28:28Z	credit_balance	TERM-MAC-001	89FC2006	1000	—
2026-05-29T12:28:18Z	credit_balance	TERM-MAC-001	89FC2006	100	—
2026-05-29T12:28:12Z	debit_card	TERM-MAC-001	89FC2006	100	—
2026-05-29T12:28:11Z — ok	authorize	TERM-MAC-001	89FC2006	100	Да
2026-05-29T12:28:00Z	register_card	TERM-MAC-001	89FC2006	1000	—
2026-05-29T12:12:10Z	credit_balance	TERM-MAC-001	89FC2006	1000	—
2026-05-29T12:12:01Z	credit_balance	TERM-MAC-001	89FC2006	100	—
2026-05-29T12:11:53Z	debit_card	TERM-MAC-001	89FC2006	100	—

Рисунок 3.1 – Результат ручного теста

Основные сценарии прошли успешно. Ошибки чтения карты выводятся в статусной строке терминала через `nfc_last_error()`.

4 Заключение

В результате лабораторных работ получился работающий прототип:

- сервер авторизации на Go с SQLite, JWT и Swagger-документацией;
- веб-админка на React/MUI с CRUD и журналом терминала;
- macOS-терминал на Flutter с FFI-доступом к PN532 через libnfc;
- формат хранения данных на MIFARE Classic с AES-шифрованием блоков.

Система запускается через Docker Compose и показывает полный цикл: запись карты, авторизация платежа, изменение баланса на носителе и появление событий в веб-интерфейсе.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Documentation — The Go Programming Language. — дата обращения: 11.06.2026. <https://go.dev/doc/>.
2. Docker Documentation. — дата обращения: 11.06.2026. <https://docs.docker.com/>.
3. SQLite: About. — дата обращения: 11.06.2026. <https://www.sqlite.org/about.html>.
4. RFC 7519: JSON Web Token (JWT). — дата обращения: 11.06.2026. <https://www.rfc-editor.org/rfc/rfc7519>.
5. libnfc: Near Field Communication (NFC) library. — дата обращения: 11.06.2026. <https://github.com/nfc-tools/libnfc>.
6. MIFARE Classic 1K: Memory Layout. — дата обращения: 11.06.2026. <https://github.com/ikarus23/MifareClassicTool/blob/master/README.md>.
7. Adafruit PN532 RFID/NFC Breakout and Shield. — дата обращения: 11.06.2026. <https://learn.adafruit.com/adafruit-pn532-rfid-nfc>.
8. OpenSSL: Cryptography and SSL/TLS Toolkit. — дата обращения: 11.06.2026. https://www.openssl.org/docs/manmaster/man3/EVP_EncryptInit.html.
9. swaggo/swag: Swagger documentation generator for Go. — дата обращения: 11.06.2026. <https://github.com/swaggo/swag>.
10. Material UI: React component library. — дата обращения: 11.06.2026. <https://mui.com/>.
11. React: The library for web and native user interfaces. — дата обращения: 11.06.2026. <https://react.dev/>.
12. Dart FFI: binding to native C APIs. — дата обращения: 11.06.2026. <https://dart.dev/guides/libraries/c-interop>.